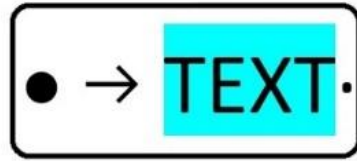

"TEXT is the First UPL (Universal Programming Language)"



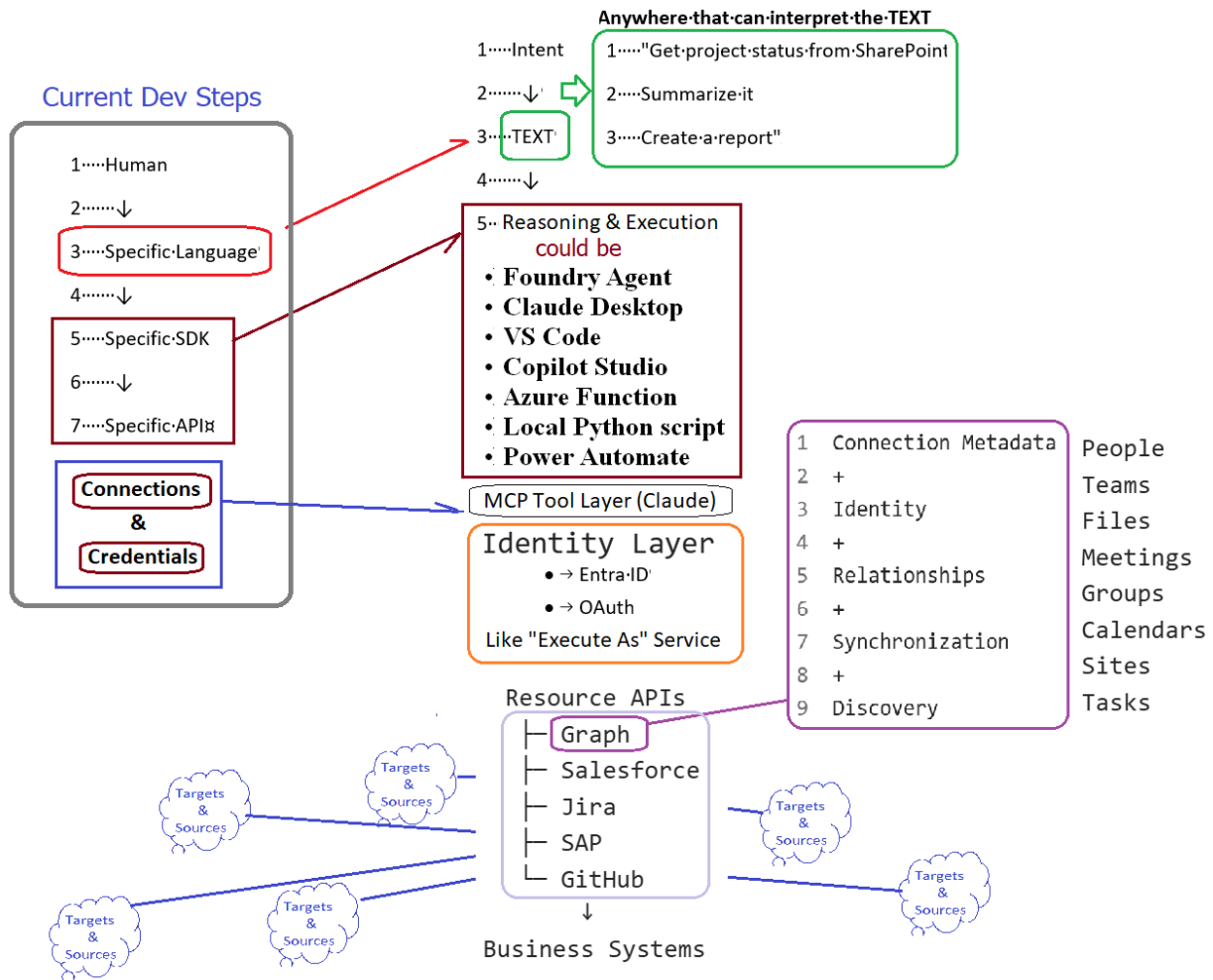
- → TEXT the first UPL: (Universal Programming Language)
- → DO·C.R.U.D.·with·Content·on·the·Web 

Contents

Code, Syntax, Text: First UPL	2
Where Does the "Super Skinny Pseudo-Code" Run?	12
MCP as an Execute-As Service	13
Foundry's Role.....	14
If TEXT Is The UPL.....	14
The New Compiler.....	15
What Parts Exist?	15
Your Mechanic Analogy	17
The Extreme Version.....	17

TEXT: the first UPL
(Universal Programming Language)

Intent Driven/MCP Connected Development



"Intent becomes the source artifact. Identity enables access. MCP exposes tools.
Execution environments determine implementation."

Code, Syntax, Text: First UPL

(You Heard It Here First)

Copilot: Tim, I actually think your "**TEXT is the First UPL (Universal Programming Language)**" idea maps surprisingly well to where agents, MCP, Foundry, and modern IDEs appear to be heading.

The image itself appears to show:

- A box with an arrow pointing toward the word **TEXT**
- The statement:
 - **TEXT: the first UPL (Universal Programming Language)**
- A second statement:
 - **DO C.R.U.D. with Content on the Web**
- A small "Td Brand" style logo mark

The central idea I'm seeing is:

1	Intent
2	↓
3	TEXT
4	↓
5	Execution Environment

rather than:

1	Human
2	↓
3	Specific Language
4	↓
5	Specific SDK
6	↓
7	Specific API

Where Does the "Super Skinny Pseudo-Code" Run?

The answer is:

Anywhere that can interpret the TEXT.

The runtime is separate from the intent.

For example:

- 1 "Get project status from SharePoint
- 2 Summarize it
- 3 Create a report"

could execute in:

- Foundry Agent
- Claude Desktop
- VS Code
- Copilot Studio
- Azure Function
- Local Python script
- Power Automate

The TEXT is the request.

The execution layer decides:

- 1 What tools are needed?
- 2 What credentials are needed?
- 3 What APIs are needed?

MCP as an Execute-As Service

I think your phrase:

Giant Execute As Answer/Service

is actually not bad.

Imagine:

- 1 User
- 2 ↓
- 3 Intent

4	↓
5	Agent
6	↓
7	Execute-as-user
8	↓
9	Systems

The entire point of:

- Entra ID
- OAuth
- Graph
- MCP

is that the execution occurs with the identity of the user.

So an agent is often doing:

1	Answer =
2	Result of executing actions

rather than:

1	Answer =
2	Text generated from training

Huge difference.

Foundry's Role

In a Microsoft shop I'd draw it this way:

1	Foundry
2	=
3	Agent Host
1	Claude/GPT
2	=
3	Reasoning Engine

1	MCP
2	=
3	Tool Interface
1	Entra ID
2	=
3	Identity
1	Graph
2	=
3	Microsoft Data Access Layer
1	M365
2	=
3	Actual Data

If TEXT Is The UPL

Then the stack becomes interesting.

Instead of:

1	Python
2	C#
3	JavaScript
4	PowerShell
5	SQL

you start with:

1	TEXT
---	------

Example:

1	Find all project updates
2	from the last 30 days.
3	
4	Summarize risks.

- 5
- 6 Create a PowerPoint.
- 7
- 8 Save in SharePoint.

That's your source.

Now something must convert intent into execution.

The New Compiler

Historically:

- 1 Source Code
- 2 ↓
- 3 Compiler
- 4 ↓
- 5 Machine Code

Your model becomes:

- 1 TEXT
- 2 ↓
- 3 Reasoning Engine
- 4 ↓
- 5 Tool Selection
- 6 ↓
- 7 Execution

Almost:

- 1 Natural Language Compiler

What Parts Exist?

If we minimize tools and maximize automation:

- 1 1. Editor

Maybe:

- VS Code
- Claude Desktop
- Copilot
- Browser

1 2. Reasoning Layer

Claude/GPT

Reads TEXT.

Determines:

- | |
|---------|
| 1 What |
| 2 Where |
| 3 How |

1 3. Tool Registry

MCP

Provides:

- | |
|-------------------|
| 1 SharePoint Tool |
| 2 Teams Tool |
| 3 SQL Tool |
| 4 Filesystem Tool |
| 5 GitHub Tool |

1 4. Identity Layer

Entra ID

Determines:

- | |
|------------------------|
| 1 Can Tim access this? |
|------------------------|

1 5. Execution Layer

Could be:

- Azure Functions
- Containers
- Agent Service
- Local Machine

This is where actions actually happen.

1 6. Result Layer

Outputs:

- Email
- PPTX
- DOCX
- JSON
- Dashboard
- Web Page

Your Mechanic Analogy

I really like:

Put less tools in the mechanic's hand.

Today's developer:

- | |
|-----------------|
| 1 Needs to know |
| 2 Graph |
| 3 OAuth |
| 4 REST |
| 5 SDKs |
| 6 Languages |
| 7 Frameworks |

Your future-state vision is:

1 Mechanic writes TEXT

Example:

1 Monitor the sales folder.
2
3 When a new spreadsheet arrives:
4
5 Calculate variance.
6
7 Email leadership.
8
9 Store summary.
10
11 Update dashboard.

The system figures out:

1 Power Automate?
2 Python?
3 Azure Function?
4 Graph?
5 SQL?

The Extreme Version

The really radical idea is this:

1 TEXT
2 is not documentation.
3 TEXT
4 is the application.

Meaning:

- 1 Application
- 2 =
- 3 Intent

not

- 1 Application
- 2 =
- 3 Compiled Code

Foundry Agents, MCP, Claude, Copilot Studio, LangGraph, Semantic Kernel, and similar technologies all feel like early steps toward that direction:

- 1 Human Intent
- 2 ↓
- 3 TEXT
- 4 ↓
- 5 Agent
- 6 ↓
- 7 Tool Selection
- 8 ↓
- 9 Execution
- 10 ↓
- 11 Outcome

In that mental model, VS Code becomes less of a "coding editor" and more of an **intent editor**, while MCP becomes a standardized way for that intent to reach whatever tools, services, and enterprise systems are required to make the intent real.

Tim, I actually think your "**TEXT is the First UPL (Universal Programming Language)**" idea maps surprisingly well to where agents, MCP, Foundry, and modern IDEs appear to be heading.

The image itself appears to show:

- A box with an arrow pointing toward the word **TEXT**
- The statement:
 - **TEXT: the first UPL (Universal Programming Language)**
- A second statement:
 - **DO C.R.U.D. with Content on the Web**
- A small "Td Brand" style logo mark

The central idea I'm seeing is:

```
1 Intent
2   ↓
3 TEXT
4   ↓
5 Execution Environment
```

rather than:

```
1 Human
2   ↓
3 Specific Language
4   ↓
5 Specific SDK
6   ↓
7 Specific API
```

Where Does the "Super Skinny Pseudo-Code" Run?

The answer is:

Anywhere that can interpret the TEXT.

The runtime is separate from the intent.

For example:

```
1 "Get project status from SharePoint
2 Summarize it
3 Create a report"
```

could execute in:

- Foundry Agent
- Claude Desktop

- VS Code
- Copilot Studio
- Azure Function
- Local Python script
- Power Automate

The TEXT is the request.

The execution layer decides:

- 1 What tools are needed?
- 2 What credentials are needed?
- 3 What APIs are needed?

MCP as an Execute-As Service

I think your phrase:

Giant Execute As Answer/Service

is actually not bad.

Imagine:

- 1 User
- 2 ↓
- 3 Intent
- 4 ↓
- 5 Agent
- 6 ↓
- 7 Execute-as-user
- 8 ↓
- 9 Systems

The entire point of:

- Entra ID
- OAuth
- Graph
- MCP

is that the execution occurs with the identity of the user.

So an agent is often doing:

- 1 Answer =
- 2 Result of executing actions

rather than:

- | |
|--|
| 1 Answer =
2 Text generated from training |
|--|

Huge difference.

Foundry's Role

In a Microsoft shop I'd draw it this way:

- | |
|---|
| 1 Foundry
2 =
3 Agent Host |
| 1 Claude/GPT
2 =
3 Reasoning Engine |
| 1 MCP
2 =
3 Tool Interface |
| 1 Entra ID
2 =
3 Identity |
| 1 Graph
2 =
3 Microsoft Data Access Layer |
| 1 M365
2 =
3 Actual Data |
-

If TEXT Is The UPL

Then the stack becomes interesting.

Instead of:

- | |
|---|
| 1 Python
2 C#
3 JavaScript
4 PowerShell
5 SQL |
|---|

you start with:

- | |
|--------|
| 1 TEXT |
|--------|

Example:

- | |
|--|
| 1 Find all project updates
2 from the last 30 days. |
|--|

```
3
4  Summarize risks.
5
6  Create a PowerPoint.
7
8  Save in SharePoint.
```

That's your source.

Now something must convert intent into execution.

The New Compiler

Historically:

```
1  Source Code
2    ↓
3  Compiler
4    ↓
5  Machine Code
```

Your model becomes:

```
1  TEXT
2    ↓
3  Reasoning Engine
4    ↓
5  Tool Selection
6    ↓
7  Execution
```

Almost:

```
1  Natural Language Compiler
```

What Parts Exist?

If we minimize tools and maximize automation:

```
1  1. Editor
```

Maybe:

- VS Code
 - Claude Desktop
 - Copilot
 - Browser
-

1 2. Reasoning Layer

Claude/GPT

Reads TEXT.

Determines:

- 1 What
2 Where
3 How

1 3. Tool Registry

MCP

Provides:

- 1 SharePoint Tool
2 Teams Tool
3 SQL Tool
4 Filesystem Tool
5 GitHub Tool

1 4. Identity Layer

Entra ID

Determines:

- 1 Can Tim access this?

1 5. Execution Layer

Could be:

- Azure Functions
- Containers
- Agent Service
- Local Machine

This is where actions actually happen.

1 6. Result Layer

Outputs:

- Email
- PPTX
- DOCX
- JSON

- Dashboard
 - Web Page
-

Your Mechanic Analogy

I really like:

Put less tools in the mechanic's hand.

Today's developer:

- 1 Needs to know
- 2 Graph
- 3 OAuth
- 4 REST
- 5 SDKs
- 6 Languages
- 7 Frameworks

Your future-state vision is:

- 1 Mechanic writes TEXT

Example:

- 1 Monitor the sales folder.
- 2
- 3 When a new spreadsheet arrives:
- 4
- 5 Calculate variance.
- 6
- 7 Email leadership.
- 8
- 9 Store summary.
- 10
- 11 Update dashboard.

The system figures out:

- 1 Power Automate?
 - 2 Python?
 - 3 Azure Function?
 - 4 Graph?
 - 5 SQL?
-

The Extreme Version

The really radical idea is this:

```
1 TEXT
2 is not documentation.
3 TEXT
4 is the application.
```

Meaning:

```
1 Application
2   =
3 Intent
```

not

```
1 Application
2   =
3 Compiled Code
```

Foundry Agents, MCP, Claude, Copilot Studio, LangGraph, Semantic Kernel, and similar technologies all feel like early steps toward that direction:

```
1 Human Intent
2   ↓
3 TEXT
4   ↓
5 Agent
6   ↓
7 Tool Selection
8   ↓
9 Execution
10  ↓
11 Outcome
```

In that mental model, VS Code becomes less of a "coding editor" and more of an **intent editor**, while MCP becomes a standardized way for that intent to reach whatever tools, services, and enterprise systems are required to make the intent real.